AIM- To develop a basic FreeRTOS-based project on STM32F446RE in STM32CubeIDE and validate LED blinking using a single RTOS task.

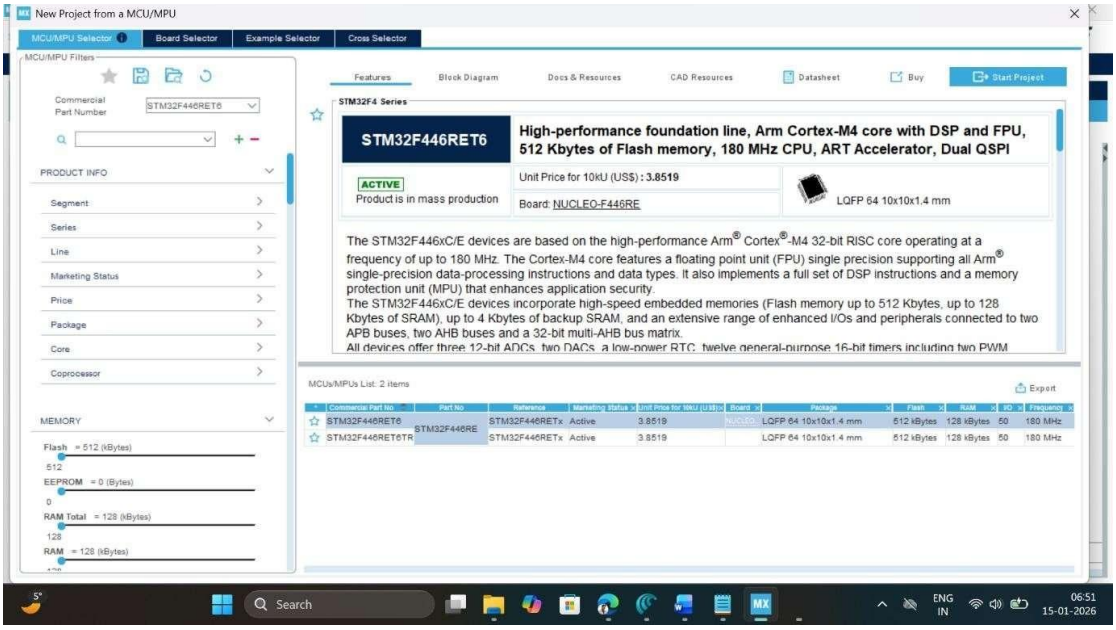
APPARATUS- STM32 Nucleo-F446RE development board, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

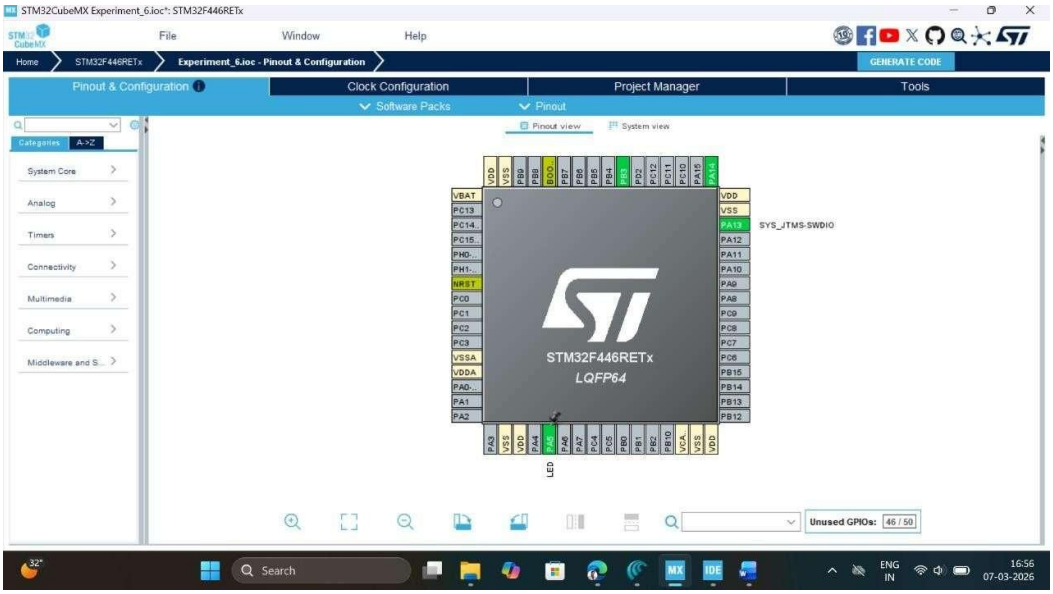
A Real-Time Operating System (RTOS) such as FreeRTOS replaces the conventional superloop architecture with a preemptive scheduler that manages multiple independent tasks, each having its own stack, priority, and execution state. In this experiment STM32F446RE uses CMSIS-RTOS v2, a single FreeRTOS task (`Task1_function`) is created via `osThreadNew()` to periodically toggle the onboard LED (PA5); the task calls `osDelay(500)` to block itself for 500 ms after each toggle, yielding CPU control back to the scheduler and creating a precise 500 ms blink period while keeping the processor available for future tasks. Standard `printf` statements within the task are redirected through a custom `_write()` function to the SWV ITM Data Console via ITM stimulus port 0, enabling non-intrusive real-time observation of task execution alongside the visible LED blinking, thus demonstrating both RTOS task management and embedded debugging techniques fundamental to Cortex-M4 development.

PROCEDURE

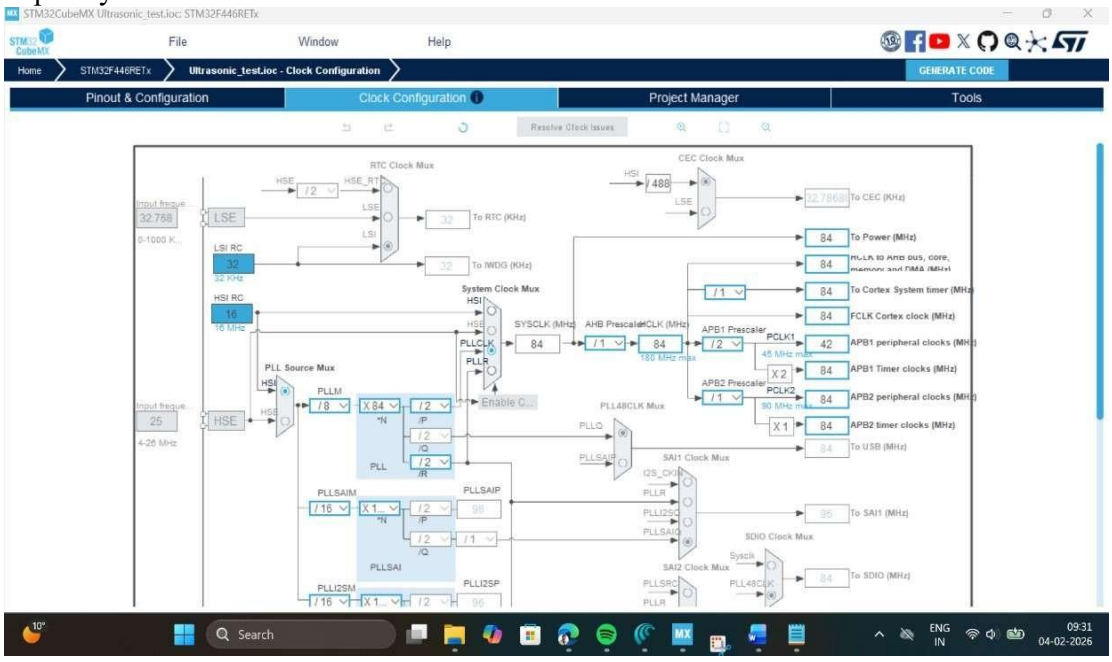
1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.



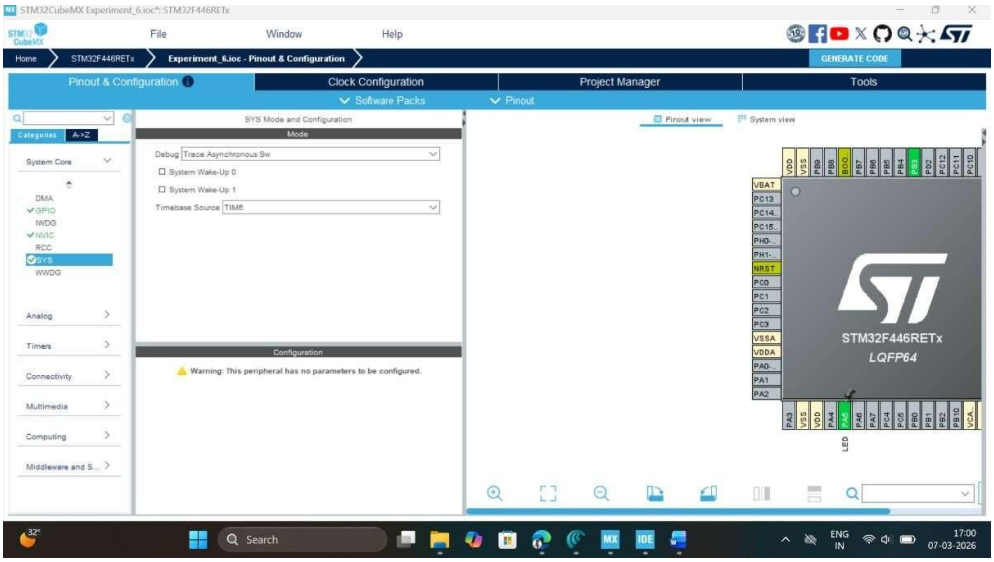
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 to make it as GPIO output pin as shown below as LED (Inbuilt LED).



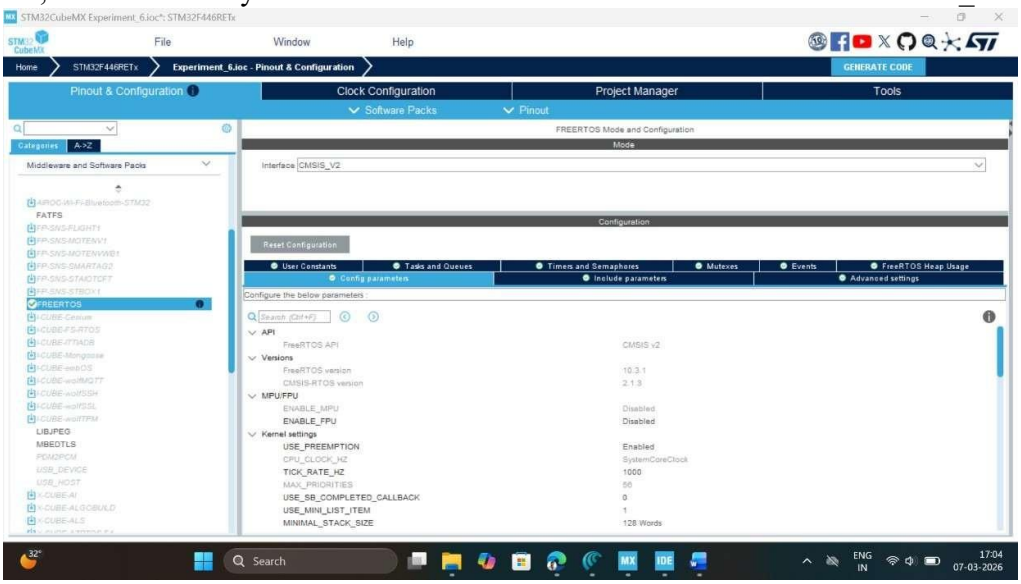
4. Clock Configuration: Configure clock as shown below to get maximum internal clock frequency of 84MHz.



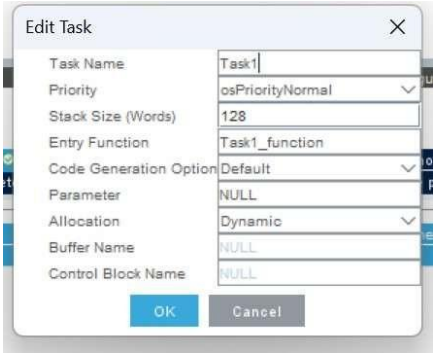
5. SYS Mode: Click on SYS mode and select debug as Trace Asynchronous Sw, and Timebase source to TIM6.



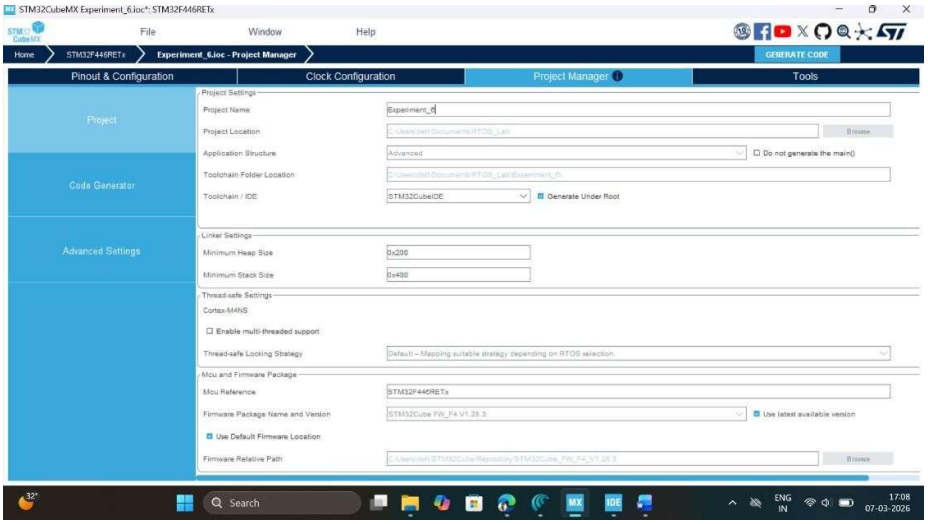
6. FreeRTOS Configuration: Click on Middleware and Software Pack, then select FreeRTOS software pack, which is already available with STMCube Mx. Select interface as CMSIS_V2.



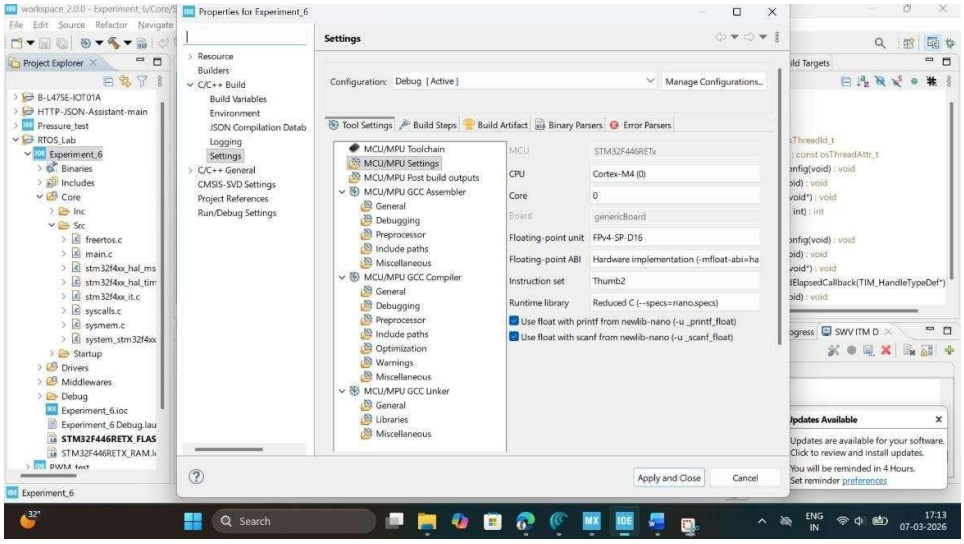
7. Click on Task & Queue Tab of configuration setting and create a Task_1 as shown below:



8. Write Project name and select IDE.

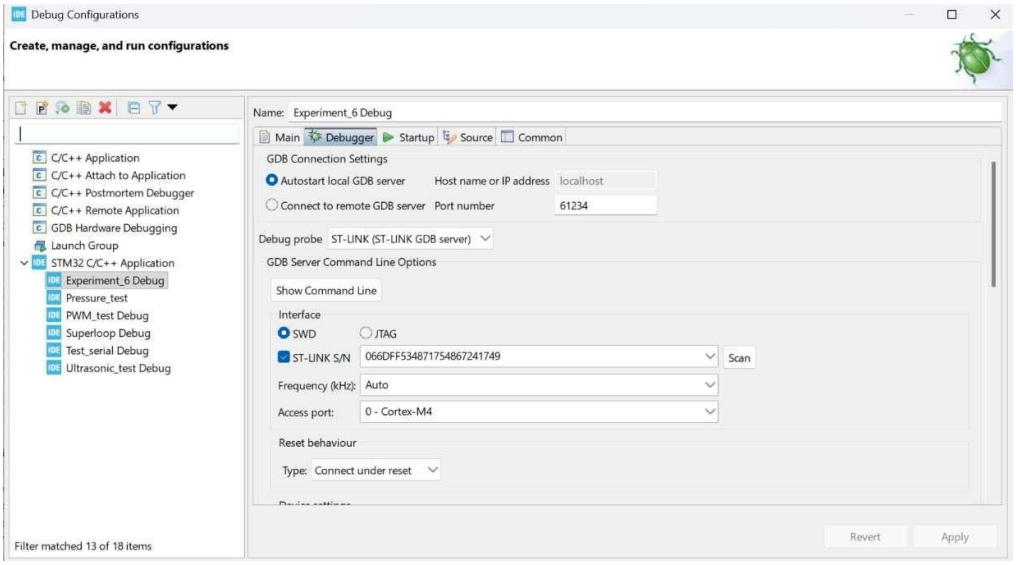


9. Click on generate code.
10. Click on open project.
11. Click ‘OK’ on the generated popup showing “successfully imported the project on the workspace” and the project will be successfully added to the cube IDE.
12. Open the main source code on Cube IDE by clicking on main.c
13. Click on project properties, go to C/C++ Build settings check box for printf and scanf settings.



14. Now click on Debug icon and configure debugger for SWV ITM Trace
15.

16. Click on Debugger configuration, select debugger console, and click on ST-LINK S/N and then scan, as we attach a STM32F446RE development board, it will automatically pick up STLINK id.



17. Next, select the Serial Wire Viewer (SWV) and check box on enable button and make sure to pass on the right core clock frequency which is set-up at per point 4 here i.e. 84MHz. Then click on Apply button and close this window.
18. After configuration write following instructions in the main.c file and compile the program and ensure there will be zero error after compiling.

```
/* USER CODE BEGIN Includes */

#include <stdio.h>

/* USER CODE END Includes */
```

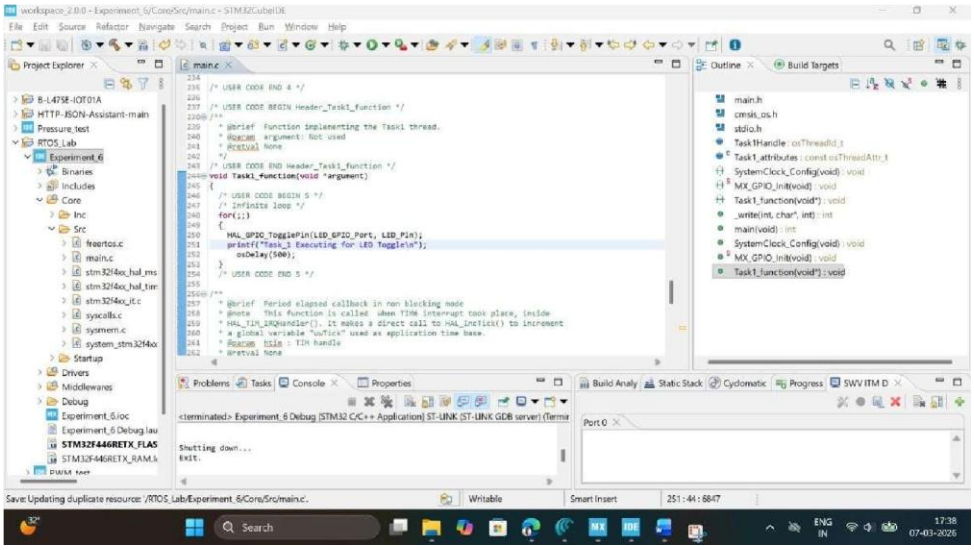
Write the function to enable SWV ITM Console to trace the output of Task_1

```
/* USER CODE BEGIN 0 */
 // Retarget fwrite to ITM  int _write(int file,
char *ptr, int len) {      for (int i = 0; i <
len; i++) {
       ITM_SendChar(*ptr++);
   }      return
len;  }
/* USER CODE END 0 */
```

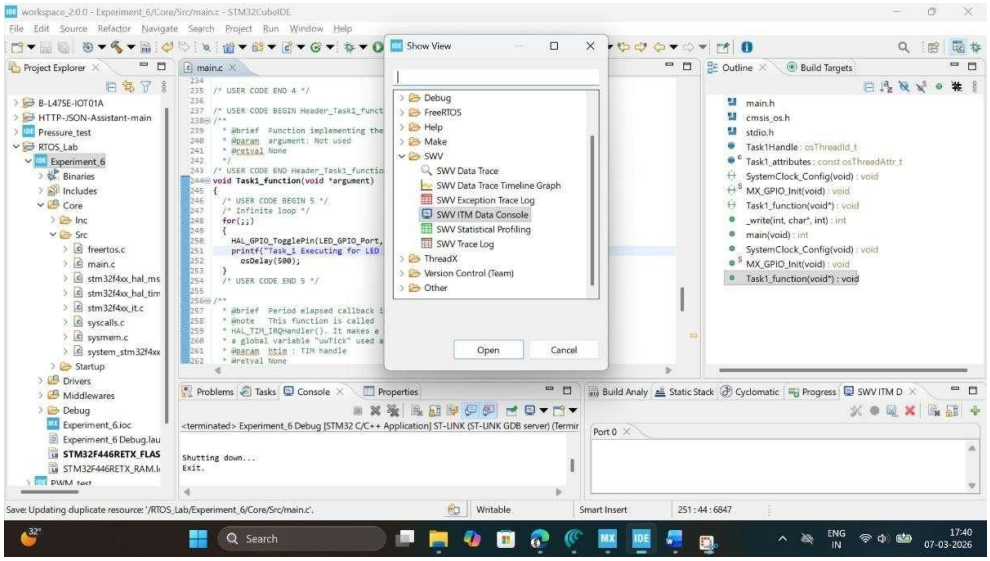
Edit the Task_1 function as given below to toggle the inbuilt LED for 500ms delay.

```
* USER CODE END Header_Task1_function */ void Task1_function(void
*argument)
{
/* USER CODE BEGIN 5 */
/* Infinite loop */ for(;;)
{
HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin); printf("Task_1 Executing
for LED Toggle \n"); osDelay(500);
```

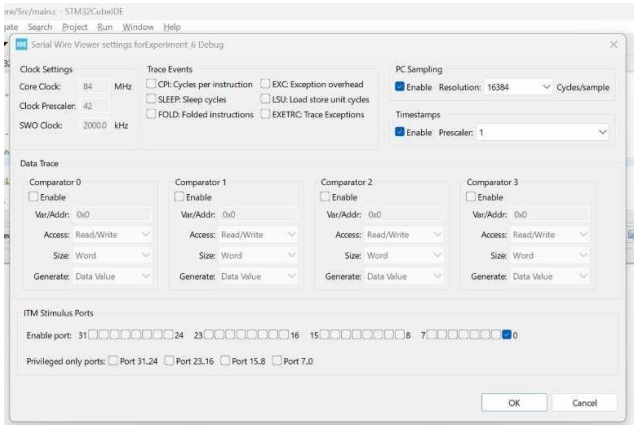
```
}
/* USER CODE END 5 */
```



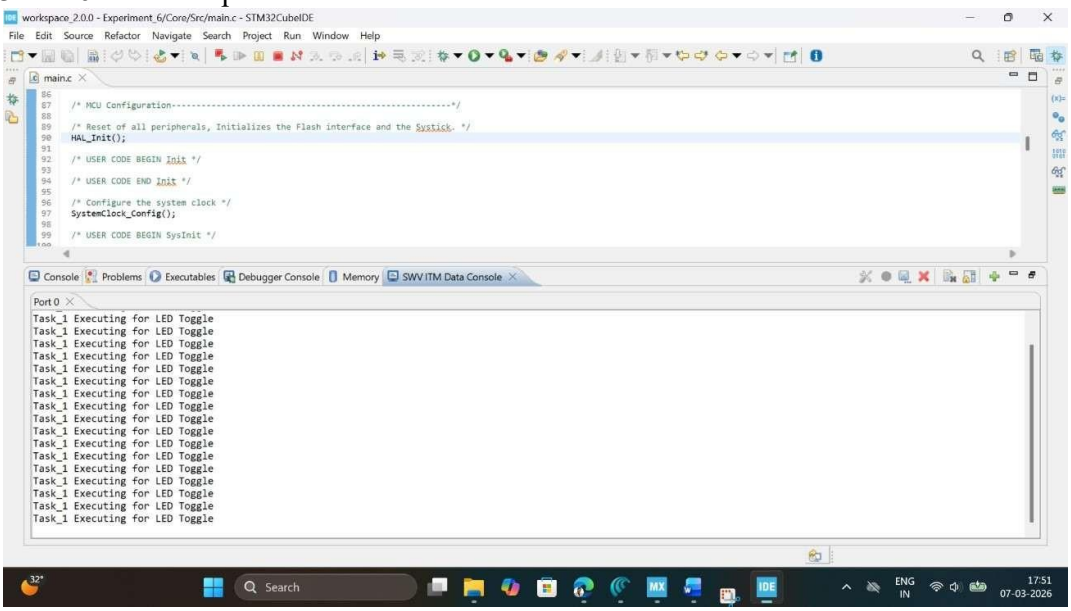
19. Now, click on Window tab, select show view ▯other ▯ select SWV ITM Data Console.



20. Now click on Debug button, start debugging this main.c file and click on switch to change the perspective to debugger.
21. Click on SWV ITM Data Console setting, enable the PC sampling and select port 0.



22. Click on Start Trace (Red Dot) on SWV ITM Data Console tab and then click on resume button to start executing the main program. Now you can see the message on Port 0 and LED Toggle on STM32F446RE development board.



OBSERVATION:

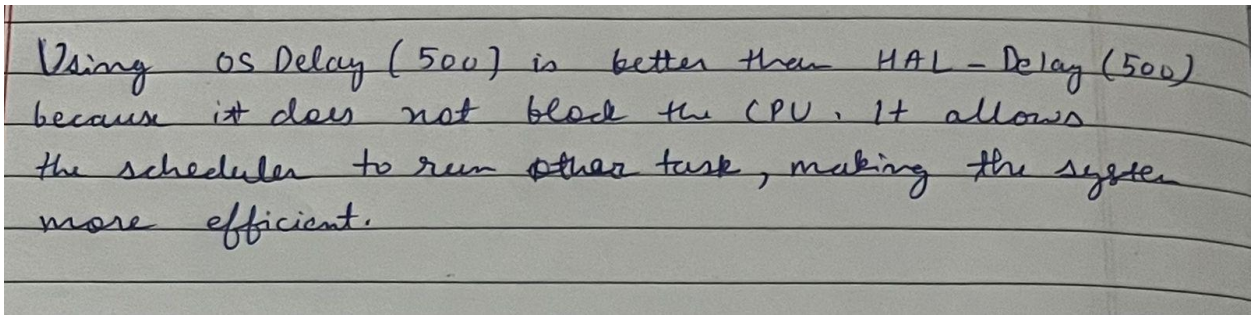
Observation: During the experiment, the onboard LED (PA5) on the STM32F446RE board blinked regularly with an observed period of 500 ms, confirming that the FreeRTOS Task_1 executed successfully with osDelay(500) providing precise 500 ms blocking intervals. Simultaneously, the SWV ITM Data Console on Port 0 displayed the message "Task_1 Executing for LED Toggle" 34 times per second, perfectly synchronized with each LED toggle event, demonstrating successful ITM retargeting via the write () function. The program compiled with 0 errors and 0 warnings, and the debugger configuration maintained stable SWV tracing throughout the 1 minute observation period without any timing drift or missed task executions, validating both the CMSIS-RTOS v2 task creation and the nonintrusive debugging setup for Cortex-M4 embedded systems.

RESULT

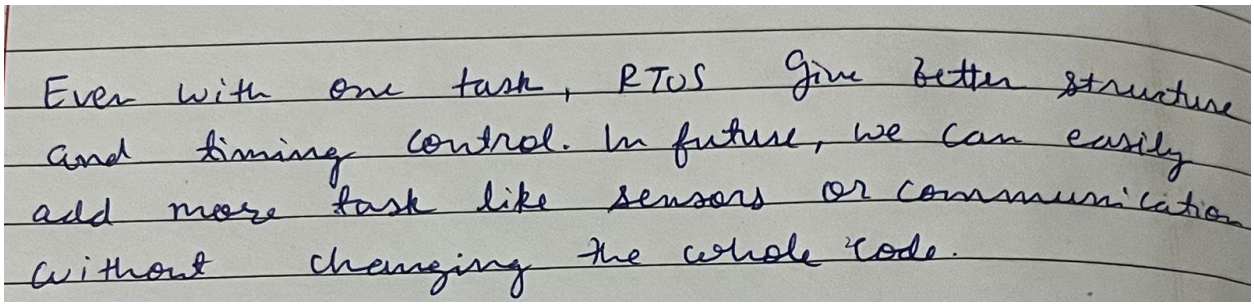
A single task (Task_1) was successfully created using FreeRTOS Middleware software pack using CMSIS V2 interface and output is observed using SWV ITM Data console Port 0 on STM Cube IDE.

Reflection Summary

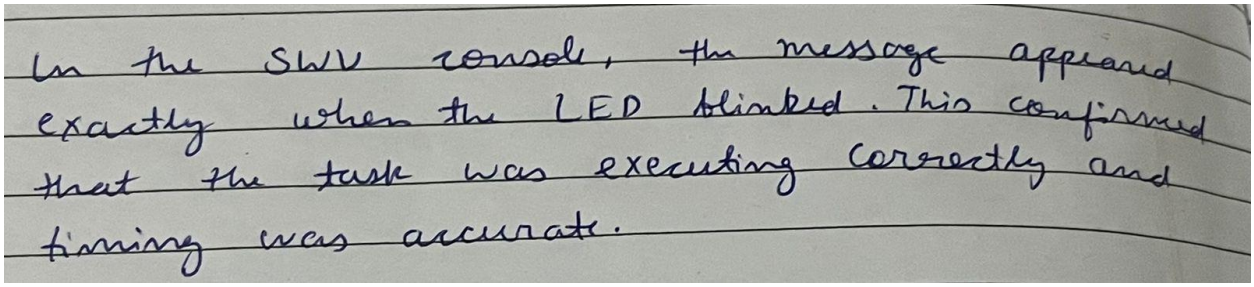
1. How does using osDelay(500) inside Task1_function differ conceptually from implementing a 500 ms LED delay using a super-loop with HAL_Delay(500) or tick-based counters?



2. In this experiment, only one task is running; what advantages does the RTOS-based approach provide now, and what additional benefits will appear when you add more tasks (e.g., UART logging, sensor reading)?



3. What did you observe in the SWV ITM Data Console when the LED was blinking, and how does this help you verify correct task timing and execution without adding extra GPIO or UART prints?



4. If the LED did not blink or no messages appeared in the SWV console, which configuration steps in the procedure (GPIO, FreeRTOS, debugger/SWV settings, _write retargeting) would you systematically re-check first, and why?